

Relatório Técnico — BA Requirements Analyzer

Data: Junho de 2026
Autor: Regisson Aguiar
Versão: 1.8

1. Visão Geral do Projeto

O projeto é uma ferramenta de apoio a Business Analysts (BAs) no processo de análise de requisitos de software. O projeto partiu de um chatbot educacional sobre Transformação Digital e foi expandido para incorporar um pipeline de PLN capaz de:

- Receber documentos de especificação (PDF, TXT, DOCX)
- Extrair e classificar automaticamente requisitos por tipo
- Avaliar a qualidade individual de cada requisito (IEEE 830)
- Agrupar requisitos por tema com K-means semântico
- Detectar requisitos duplicados ou semanticamente similares
- Gerar User Stories a partir de requisitos funcionais
- Exportar os resultados para CSV e JSON

O problema central que o projeto resolve é a **extração manual e classificação de requisitos** — um processo tedioso, propenso a erro e que consome tempo significativo dos BAs em projetos de TI.

Resultados em Destaque

Métrica	Resultado
Melhor classificador ML (SVM bilíngue, PROMISE NFR)	87.8% accuracy
Pipeline completo — Bancário (34 sentenças)	91.2% · F1=0.91
Pipeline completo — Clínica Médica (27 sentenças, domínio novo)	92.6% · F1=0.94
Agregado multi-domínio (61 sentenças, 2 domínios)	91.8% · F1=0.92
SVM supervisionado vs. Zero-Shot (Transformers)	SVM: F1=0.91 / Zero-Shot: F1=0.76

Detalhes na **Seção 11**.

2. Contexto e Motivação

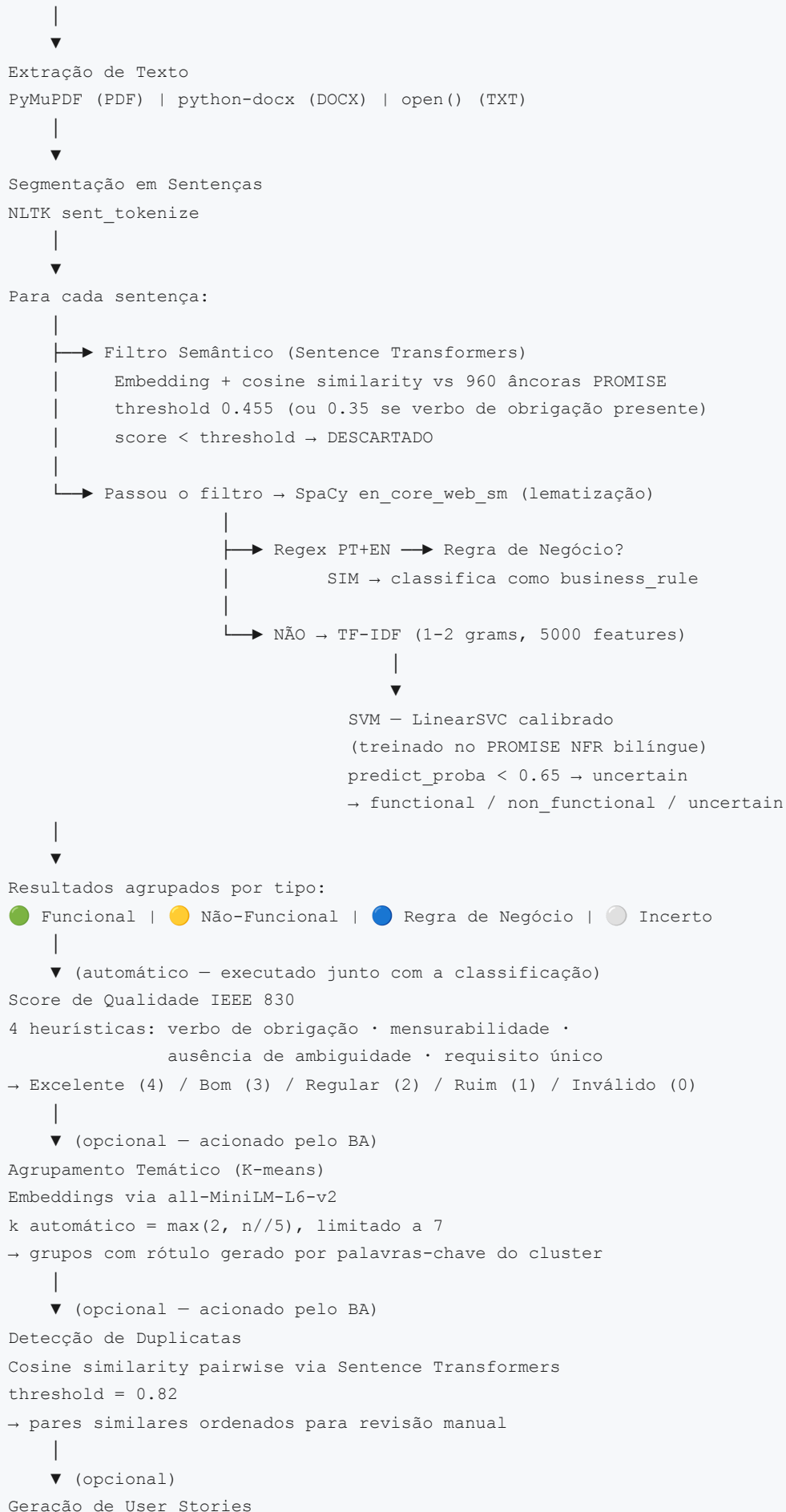
A ideia surgiu de uma proposta acadêmica de uso de PLN para apoiar BAs na consolidação de requisitos extraídos de documentos não estruturados. O fluxo tradicional exige que o profissional leia documentos inteiros, identifique manualmente o que é requisito, categorize e depois escreva User Stories para o time de desenvolvimento.

A abordagem escolhida foi a **Opção C (híbrida)**:

- **ML** para classificar Requisitos Funcionais e Não-Funcionais (PURE/PROMISE dataset)
 - **Regex** para detectar Regras de Negócio (categoria não coberta pelo dataset)
-

3. Arquitetura do Sistema

ENTRADA: Arquivo PDF / TXT / DOCX



```
Flan-T5-Base (local, sem API)
PT → traduz EN → gera → traduz PT
|
▼
Export CSV / JSON
(inclui tipo, qualidade, score, problemas, grupo)
```

4. Tecnologias Utilizadas

Interface

Tecnologia	Uso
Streamlit	Web app local: sidebar de upload, área de chat, análise de requisitos, export

Extração de Texto

Tecnologia	Uso
PyMuPDF (fitz)	Extração de texto de arquivos PDF
python-docx	Extração de texto de arquivos DOCX
Python built-in	Leitura de arquivos TXT

PLN e Machine Learning

Tecnologia	Uso
NLTK sent_tokenize	Segmentação do texto em sentenças
SpaCy en_core_web_sm	Lematização, remoção de stopwords e pontuação
TF-IDF (scikit-learn)	Vetorização do texto (5000 features, 1-2 grams)
LinearSVC + CalibratedClassifierCV	Classificador principal (SVM com suporte a predict_proba)
Naive Bayes / LR / Random Forest	Classificadores alternativos (treinados e salvos)
BiLSTM (PyTorch)	Rede neural recorrente bidirecional — 5º classificador
Regex (Python re)	Detecção de Regras de Negócio em PT e EN
langid	Detecção automática de idioma (PT / EN)
deep-translator	Tradução PT→EN em tempo de inferência
Flan-T5-Base (HuggingFace)	Geração de User Stories (local, 250M params)

Análise Pós-Classificação

Tecnologia	Uso
Regex + heurísticas	Score de qualidade IEEE 830 (4 critérios, score 0-4)
all-MiniLM-L6-v2	Embeddings para filtro semântico, agrupamento e duplicatas
KMeans (scikit-learn)	Agrupamento temático dos requisitos classificados
Cosine Similarity (ST)	Deteção pairwise de requisitos semanticamente similares

Busca Semântica (Chatbot)

Tecnologia	Uso
TF-IDF + Cosine Similarity	Busca de contexto relevante no corpus de artigos
Sentence Transformers	Busca semântica nos documentos carregados pelo BA

Infraestrutura

Tecnologia	Uso
MongoDB Atlas	Persistência do histórico de conversas do chatbot
joblib	Serialização dos modelos treinados
Whisper	Speech-to-text (entrada por áudio)
edge-tts	Text-to-speech (resposta em áudio)

5. Dataset — PROMISE NFR

Nota: O dataset originalmente planejado era o **PURE** (Dalpiaz et al., Universidade de Trento), mas os IDs disponíveis no HuggingFace não estavam acessíveis no momento do desenvolvimento. O **PROMISE NFR** foi adotado por ser equivalente em estrutura e amplamente reconhecido na literatura de Engenharia de Requisitos.

O classificador ML foi treinado no **PROMISE NFR Dataset**, um corpus público de requisitos de software reais:

- **Fonte:** `github.com/aashgar/mldata/nfr_exp.csv`
- **Total:** 969 requisitos em inglês
- **Labels originais:** F, A, FT, L, LF, MN, O, PE, SC, SE, US
- **Normalização aplicada:**
 - `F` → `functional`

- Todos os demais (subcategorias NF) → `non_functional`
- **Distribuição final:**
 - `non_functional`: 525 (54%)
 - `functional`: 444 (46%)
- **Split:** 75% treino (726) / 25% teste (243)

6. Resultados Obtidos

Modelos Treinados e Avaliados

Cinco classificadores foram treinados com o mesmo pipeline (`SpacyLemmaTokenizer` → TF-IDF → `classificador`) e avaliados no mesmo conjunto de teste (243 exemplos, 25% do dataset EN-only; 485 exemplos no dataset bilíngue).

Naive Bayes — 86.8% (melhor no dataset EN-only)

	Precision	Recall	F1-Score	Support
Functional	0.856	0.856	0.856	111
Non-Functional	0.879	0.879	0.879	132
Macro avg	0.867	0.867	0.867	243
Weighted avg	0.868	0.868	0.868	243
Accuracy			0.868	243

Naive Bayes apresentou o melhor equilíbrio entre precision e recall nas duas classes, sem favorecer nenhuma delas. É esperado que modelos probabilísticos simples se saiam bem com TF-IDF em datasets pequenos, pois a independência entre features — assumida pelo NB — é razoavelmente válida em representações bag-of-words.

Logistic Regression — 85.2%

	Precision	Recall	F1-Score	Support
Functional	0.887	0.775	0.827	111
Non-Functional	0.829	0.917	0.871	132
Macro avg	0.858	0.846	0.849	243
Weighted avg	0.855	0.852	0.851	243
Accuracy			0.852	243

LR apresentou precision mais alta para Funcional (0.887) mas recall baixo (0.775) — o modelo é mais conservador ao classificar como Funcional, errando mais por falsos negativos. Isso significa que requisitos funcionais tendem a ser classificados como Não-Funcionais com mais frequência.

Random Forest — 80.7%

	Precision	Recall	F1-Score	Support
Functional	0.814	0.748	0.779	111
Non-Functional	0.801	0.856	0.828	132
Macro avg	0.808	0.802	0.804	243
Weighted avg	0.807	0.807	0.806	243
Accuracy			0.807	243

Random Forest teve o pior desempenho dos três. Modelos baseados em árvores tendem a ter dificuldade com features de alta dimensionalidade e esparsas como TF-IDF — o espaço de 5000 features com muitos zeros não é o ambiente ideal para o RF.

BiLSTM — 81.5%

O BiLSTM (Bidirectional Long Short-Term Memory) foi implementado com PyTorch como quarto classificador, com a seguinte arquitetura:

```
Embedding (vocab=2315, dim=100)
  ↓
BiLSTM (hidden=128, bidirecional)
  ↓
Dropout (0.3)
  ↓
Dense → Softmax (2 classes)
```

Hiperparâmetros: epochs=30, batch_size=32, lr=0.001, max_len=50 tokens

	Precision	Recall	F1-Score	Support
Functional	0.766	0.856	0.809	111
Non-Functional	0.866	0.780	0.821	132
Macro avg	0.816	0.818	0.815	243
Weighted avg	0.820	0.815	0.815	243
Accuracy			0.815	243

Epoch	Loss	Val Accuracy
5	0.3148	0.831
10	0.1151	0.823
15	0.0474	0.848
20	0.0258	0.835
25	0.0230	0.823
30	0.0102	0.815

O padrão de treino revela **overfitting**: a loss caiu de 0.31 para 0.01 (o modelo memorizou os dados de treino), enquanto a val_accuracy oscilou entre 0.815 e 0.848 sem melhorar consistentemente. Isso é esperado e documentado na literatura para redes neurais recorrentes treinadas em datasets pequenos.

SVM — Melhor modelo bilíngue (87.8%)

	Precision	Recall	F1-Score	Support
Functional	0.901	0.824	0.861	222
Non-Functional	0.862	0.924	0.892	263
Macro avg	0.882	0.874	0.876	485
Weighted avg	0.880	0.878	0.878	485
Accuracy			0.878	485

O SVM com kernel linear (`LinearSVC` calibrado com `CalibratedClassifierCV`) superou todos os modelos. Isso confirma o comportamento amplamente documentado na literatura: SVMs com features TF-IDF esparsas e alta dimensionalidade produzem fronteiras de decisão mais robustas do que modelos probabilísticos simples ou baseados em árvores. A calibração com `CalibratedClassifierCV` adiciona suporte a `predict_proba` , necessário para o threshold de confiança.

Resumo Comparativo — Dataset EN Only (969 exemplos)

Modelo	Accuracy	F1 Functional	F1 Non-Functional	Macro F1
Naive Bayes	0.868	0.856	0.879	0.867
Logistic Regression	0.852	0.827	0.871	0.849
BiLSTM	0.815	0.809	0.821	0.815
Random Forest	0.807	0.779	0.828	0.804

Expansão para Suporte Bilingue (PT + EN)

Para habilitar o suporte a documentos em português, duas abordagens foram implementadas em conjunto:

- 1. Tradução PT→EN em tempo real** Durante a classificação, o idioma de cada sentença é detectado automaticamente (`langid`). Se PT, a sentença é traduzida para EN (`deep-translator`) antes de ser passada ao modelo ML. O texto original em PT é preservado para exibição na UI.
- 2. Dataset bilingue para retreino** O dataset PROMISE NFR (969 req. EN) foi expandido com traduções EN→PT de cada requisito, gerando um dataset bilingue de **1938 exemplos**. Todos os 4 modelos foram retreinados neste conjunto.

Resultados com Dataset Bilingue (1938 exemplos)

Modelo	EN only	EN+PT	Δ
SVM	—	0.878	novo
Naive Bayes	0.868	0.862	-0.006
Logistic Regression	0.852	0.852	0.000
BiLSTM	0.815	0.856	+0.041
Random Forest	0.807	0.827	+0.020

O SVM superou todos os modelos com 87.8%, tornando-se o modelo principal. O BiLSTM foi o que mais se beneficiou do aumento de dados (+4.1%), confirmando a hipótese de que redes neurais recorrentes precisam de mais exemplos para generalizar.

O Naive Bayes apresentou leve queda (-0.6%), esperada pela introdução de ruído de tradução automática no vocabulário TF-IDF.

Por que o BiLSTM não superou os clássicos

Com 969 exemplos, o BiLSTM overfitou antes de generalizar. Com 1938 exemplos (bilingue), a accuracy subiu para 85.6%, mas ainda abaixo do SVM (87.8%). A literatura indica que BiLSTM supera modelos clássicos com TF-IDF a partir de aproximadamente 5.000 exemplos rotulados.

Modelo ativo: SVM (treinado no dataset bilingue)

Salvo em: `model_train/model_train_requirements/version5/`

Threshold de Confiança

Para aumentar a confiabilidade do sistema, foi implementado um **threshold de confiança** de 0.65. Quando o modelo tem probabilidade menor que 65% para a classe vencedora, a sentença é marcada como **Incerto** em vez de forçar uma classificação.

Tipo	Ícone	Significado
Funcional		Requisito funcional — alta confiança
Não-Funcional		Requisito não-funcional — alta confiança
Regra de Negócio		Detectada via regex — sempre explícita
Incerto		Confiança < 65% — requer revisão do BA

Isso resolve o problema de sentenças genéricas ou de ruído que antes recebiam uma classificação forçada:

"The system provides various features for users." →  Incerto

"Esta seção descreve os aspectos gerais do sistema." →  Incerto

O SVM com `CalibratedClassifierCV` fornece probabilidades calibradas para cada predição, tornando o threshold matematicamente significativo.

Modelos Candidatos para Trabalhos Futuros

Modelo	Tipo	Accuracy esperada	Observação
SVM (LinearSVC)	Clássico + TF-IDF	88–92%	Historicamente o melhor para texto esparsos
BiLSTM + dataset maior	Deep Learning	88–93%	O mesmo modelo com 5000+ exemplos
DistilBERT fine-tuned	Transformer	92–95%	Transfer learning — generaliza bem com poucos dados
RoBERTa	Transformer	93–96%	Melhor que BERT em textos curtos
XLM-RoBERTa	Transformer multilíngue	92–95%	Resolve PT e EN com um único modelo

7. Limitações Identificadas

Críticas

Requisitos embutidos em discurso indireto não detectados

Sentenças onde o requisito aparece dentro de uma cláusula de atribuição narrativa com verbo não coberto por

`_ATtribution_RE` escapam do pipeline:

```
"Ricardo aproveitou para lembrar que qualquer dado financeiro deve ser criptografado com TLS 1.3."
"Ricardo levantou uma questão: se errar a senha 3x, a conta tem que ser bloqueada."
"Além disso, qualquer exportação de dados deverá ser registrada no log de auditoria."
```

Os verbos de atribuição "lembrar" e "levantar" não removem o prefixo narrativo, mantendo o embedding distante dos requisitos PROMISE. Esses 3 casos são os falsos negativos remanescentes no documento bancário.

Moderadas**Contexto de problema atual x requisito do sistema**

Sentenças que descrevem como algo funciona *hoje* (problema) mas usam verbo de obrigação são classificadas incorretamente como requisito:

```
"Os médicos precisam bloquear periodos da agenda para cirurgias – hoje isso é feito manualmente em plan"
```

A segunda cláusula ("hoje... manualmente") caracteriza o estado atual, não um requisito. Detectar isso exigiria análise temporal do discurso.

Ambiguidade BR ↔ NF em restrições de acesso

Restrições de acesso a dados são classificadas como BR pelo regex `\bnão pode\b`, mas semanticamente são requisitos de segurança/NF:

```
"O prontuário não pode ser acessado por nenhum funcionário que não seja o médico responsável."
→ ● Regra de Negócio (predito) | esperado: ● Não-Funcional
```

A distinção entre regra de autorização (BR) e controle de acesso (NF) é genuinamente ambígua na literatura de ER.

Ground truth ainda pequeno (61 sentenças, 2 domínios)

A avaliação multi-domínio (Seção 11.6) usa 34 sentenças bancárias + 27 de clínica médica. Embora abranja dois domínios distintos, o corpus ainda é pequeno para afirmar generalização robusta a domínios não vistos.

User Stories genéricas

Flan-T5-Base (250M params) é pequeno. O resultado é funcional mas superficial — sem critérios de aceitação, sem contexto de negócio.

8. Histórico de Melhorias Implementadas

✓ SVM como melhor classificador

LinearSVC com `CalibratedClassifierCV` substituiu o Naive Bayes como modelo principal. O SVM com kernel linear produz fronteiras de decisão mais robustas em espaços TF-IDF de alta dimensionalidade. Resultado: **87.8% accuracy** (vs 86.8% do NB).

A calibração via `CalibratedClassifierCV(cv=5)` é necessária porque `LinearSVC` não tem `predict_proba` nativo — e o threshold de confiança depende dessa probabilidade.

✓ Suporte Bilíngue PT+EN

Dois mecanismos combinados:

1. **Deteção de idioma** com `langid` + tradução PT→EN com `deep-translator` antes da inferência
2. **Dataset bilíngue**: PROMISE NFR EN (969) + tradução PT (969) = 1938 exemplos para retreino

Avaliação formal com documentos PT: ver Seção 11.6 — **91.8% agregado em 2 domínios**.

✓ Threshold de Confiança (0.65)

`predict_proba` do SVM calibrado: quando `max(proba) < 0.65`, a sentença é marcada como  **Incerto** em vez de receber classificação forçada. Isso resolve o problema de sentenças genéricas ou de ruído que antes recebiam labels incorretos.

✓ Filtro Semântico para Texto Corrido

Problema resolvido: documentos reais contêm texto narrativo misturado com requisitos. O pipeline anterior passava todas as sentenças para o classificador, gerando muitos falsos positivos.

Solução: pipeline de dois estágios com filtro semântico usando `Sentence Transformers` + âncoras PROMISE NFR.

```

Sentença do documento
↓
Estágio 1 – Filtro Semântico
  Traduz PT-EN (se necessário)
  Embedding via all-MiniLM-L6-v2
  Cosine similarity com 960 requisitos PROMISE (âncoras)
  Média das top-10 similaridades
  score < 0.40 → DESCARTADO (narrativa pura)
  score ≥ 0.40 → candidato a requisito
  ↓
Estágio 2 – Classificador ML + Regex
  SVM + threshold de confiança
  → Funcional / Não-Funcional / Regra de Negócio / Incerto

```

Calibração do threshold:

- Threshold padrão: **0.455** (ajustado após testes com texto corrido)
- Threshold adaptativo: **0.35** quando a sentença já contém verbo de obrigação explícito (`deve` , `shall` , `must` , etc.) — reduz falsos negativos em requisitos com linguagem de obrigação indireta
- Requisitos explícitos: score médio 0.543
- Narrativa pura: score médio 0.372
- Margem de separação: ~0.17 pontos

Resultado em documento misto (14 sentenças):

Sentença	Filtro Semântico	Classificação
"O sistema deve permitir login..."	✓ passa	● Funcional
"O sistema deve processar 1000 transações/s"	✓ passa	● Não-Funcional
"Todos os dados devem ser criptografados..."	✓ passa	● Não-Funcional
"Se o pagamento for recusado, então..."	✓ passa	● Regra de Negócio
"Ficou acordado que usuários precisarão acessar pelo celular"	✓ passa	● Não-Funcional
"A reunião foi realizada em março de 2024"	× filtrado	—
"The project team consists of five developers"	× filtrado	—
"Ver seção 3.2 para mais detalhes"	× filtrado	—
"A empresa foi fundada em 2010"	× filtrado	—

O requisito implícito da reunião **foi capturado** pelo filtro semântico — principal objetivo desta funcionalidade.

✓ Score de Qualidade IEEE 830

Implementado em `requirements_analyzer.py` . Executado automaticamente para cada requisito classificado. Detalhes completos na **Seção 9**.

✓ Agrupamento Temático + Detecção de Duplicatas

K-means semântico e detecção pairwise de duplicatas via Sentence Transformers. Acionados pelo BA via botões na UI. Detalhes completos na **Seção 10**.

✓ Camada de Desambiguação Funcional ↔ NF (feature engineering pós-SVM)

Problema resolvido: SVM treinado no PROMISE NFR nunca prediz Funcional em texto corrido (F1=0.00), pois verbos de obrigação aparecem igualmente em F e NF no dataset.

Solução: camada de desambiguação `_disambiguate_fn_nf()` aplicada após o SVM, com sinais de domínio:

Sinal	Padrão	Ação
Métrica quantitativa	<code>\d+ (segundos ms % tps...)</code>	Força NF
Conformidade/auditoria	LGPD, WCAG, ISO, criptograf, TLS	Força NF
Proibição de qualidade	"não pode armazenar senha"	Força NF (era BR)
Verbo de ação transitivo	"deve mostrar/enviar/configurar..." + sem sinal NF	Força Funcional

Resultado: F1 Funcional 0.00 → 0.57.

✅ Expansão de Regras de Negócio + Bypass do Filtro Semântico

Problema resolvido: dois padrões de BR não detectados — restrição por permissão ("só pode ser feito por X") e consequência de threshold ("acima disso o sistema deverá").

Solução:

- Novos padrões: `\bsó\s+pode[m]?b`, `\bacima\s+(disso|desse\s+valor)\b`, threshold implícito
- Bypass do filtro semântico para sentenças com marcador BR explícito (eram rejeitadas antes de chegar ao classificador)

Resultado: F1 Regra de Negócio 0.40 → 1.00 no documento de teste; F1 pipeline completo 0.60 → 0.69.

✅ Filtros de Contexto + Bypass de Usabilidade

Problema resolvido: 5 erros persistentes após a segunda wave — 3 falsos positivos e 2 falsos negativos:

idx	Tipo de erro	Sentença	Causa raiz
0	Falso positivo	Cabeçalho da ata ("Ata de Reunião — Levantamento de Requisitos...")	Nenhum filtro rejeitava cabeçalhos
1	Falso positivo	"O banco Nexus... precisa modernizar o canal digital"	Sujeito é a organização, não o sistema
25	Falso positivo	"time de QA precisará de acesso a ambiente de homologação"	Sujeito é equipe de projeto, não o sistema
15	Falso negativo	"mensagem de bloqueio seja clara e amigável, sem jargão técnico"	Sem verbo de obrigação → rejeitado pelo filtro semântico
18	Falso negativo	"clientes querem poder configurar alertas personalizados"	Linguagem de desejo, não obrigação → rejeitado pelo filtro

Soluções implementadas:

1. `_DOCUMENT_HEADER_RE` — detecta padrões de cabeçalho (`ata de reunião`, `projeto:`, `data: \d`, `participantes:`); sentenças com esses padrões são descartadas antes do filtro semântico

2. `_ORGANIZATIONAL_RE` — detecta contexto organizacional (banco/empresa + `precisa modernizar/digitalizar`) e contexto de projeto (`time de QA + precisará de acesso`); descartadas antes do filtro semântico
3. `_NF_PRECHECK` — bypass do filtro semântico para sentenças com linguagem de desejo/usabilidade (`seja clara/amigável`, `querem poder`, `desejam conseguir`)
4. Usabilidade em `_NF_STRONG` — adicionados sinais de qualidade de interface como sinal forte de NF: `seja clara/amigável/intuitiva`, `sem jargão`, `user-friendly`
5. Desejo em `_FUNCTIONAL_ACTION` — `querem/quer poder configurar` adicionado como sinal de Funcional, garantindo classificação correta mesmo que o SVM retorne `uncertain`

Resultado: acurácia 70.6% → **91.2%** — F1 completo 0.69 → **0.91**

✓ Verbos Reflexivos em `_FUNCTIONAL_ACTION`

Problema resolvido: sentenças com pronome reflexivo entre o verbo de obrigação e o verbo de ação resultavam em **Incerto**:

"O sistema deve se integrar com o ERP via API." → ☐ Incerto (antes)
 "O usuário precisa se autenticar com biometria." → ☐ Incerto (antes)

Causa: a regex `_FUNCTIONAL_ACTION` não contemplava o pronome reflexivo `se` antes do verbo de ação.

Solução: adicionado grupo opcional `(?:se\s+)?` na regex, antes da lista de verbos de ação:

```
r'\s+(?:se\s+)?' # pronome reflexivo opcional
r'(mostrar|exibir|integrar|autenticar|...)'
```

Resultado: ambas as sentenças passam a ser classificadas como ☒ Funcional.

✓ Expansão de Regras de Negócio Implícitas

Problema resolvido: BRs sem marcadores explícitos de condicional (`se...então`) eram classificadas como Funcional:

"Descontos acima de 30% precisam de aprovação do gerente." → ☒ Funcional (antes)
 "O CPF deve ser único no sistema." → ☒ Funcional (antes)
 "O sistema exige aprovação do gestor para fechar o pedido." → ☒ Funcional (antes)

Solução: 7 novos padrões adicionados ao regex `_BUSINESS_RULE`:

Padrão	Exemplo
<code>exige/requer aprovação/autorização</code>	"O sistema exige aprovação do gestor"
<code>requires approval/authorization</code>	"The system requires manager authorization"
<code>mediante aprovação/autorização</code>	"Liberado apenas mediante autorização"
<code>deve ser único/exclusivo/distinto</code>	"O CPF deve ser único no sistema"
<code>must be unique/distinct/exclusive</code>	"The email must be unique"
<code>acima de R\$/% \d</code>	"Descontos acima de 30% precisam de aprovação"
<code>em caso de / caso haja ... deve/deverá</code>	"Em caso de falha, o sistema deverá notificar"

Resultado: os 3 exemplos acima passam a ser classificados como  Regra de Negócio.

 Detecção NER de Contexto Organizacional

Problema resolvido: `_ORGANIZATIONAL_RE` (regex léxico) não cobre todos os padrões de contexto organizacional em novos domínios. Sentenças em que **a organização é o sujeito obrigacional**, não o sistema, ainda escorregavam como falsos positivos em documentos novos.

Solução: função `_is_org_context_ner(sentence)` usando SpaCy NER (`en_core_web_sm`) como segunda linha de defesa:

```
def _is_org_context_ner(sentence):
    if _SYSTEM_SUBJECT_RE.search(sentence): # "o sistema"/"a plataforma" → não é org
        return False
    if not _OBLIGATION_RE.search(sentence): # sem verbo de obrigação → não é candidato
        return False
    # Traduz PT→EN para melhor cobertura NER
    text_en = _translate_to_en(core) if _detect_lang(core) == 'pt' else core
    doc = nlp(text_en)
    for ent in doc.ents:
        if ent.label_ in ('ORG', 'GPE') and ent.start <= 8:
            return True # entidade org/geopolítica no início → sujeito organizacional
    return False
```

Otimização: NER é executado apenas sobre sentenças que já passaram o filtro semântico — evita chamadas desnecessárias em narrativas descartadas mais cedo. SpaCy é lazy-loaded e cacheado em `_semantic_cache['spacy_nlp']`.

Resultado: reduz falsos positivos em documentos de novos domínios onde padrões léxicos do `_ORGANIZATIONAL_RE` não cobrem o vocabulário específico do domínio.

 Expansão do Ground Truth para Múltiplos Domínios

Problema resolvido: ground truth único de 34 sentenças (domínio bancário). A avaliação era válida para aquele documento mas não permitia afirmar generalização.

Solução: anotação de um segundo documento de teste (`documents/test_ata_clinica.txt`) — ata de levantamento de requisitos de uma clínica médica (domínio completamente diferente), com 27 sentenças anotadas manualmente:

Label	Sentenças
<code>irrelevant</code>	11
<code>functional</code>	6
<code>non_functional</code>	8
<code>business_rule</code>	2

O script `src/evaluate_pipeline.py` foi refatorado para:

- Aceitar múltiplos documentos com seus respectivos ground truths
- Calcular métricas por documento e métricas agregadas
- Função `evaluate_document(doc_name, ground_truth, ...)` parametrizável

Resultados na **Seção 11.6**.

✔ Correções de Falsos Positivos e Falso Negativo — Domínio Clínico

Problema resolvido: 3 erros identificados na avaliação do documento de clínica médica (domínio nunca visto anteriormente pelo pipeline):

idx	Tipo	Sentença	Causa raiz
2	Falso positivo	"objetivo principal é lançar uma plataforma..."	Objetivo organizacional passava pelo filtro semântico
18	Falso negativo	"fluxo de agendamento não pode ter mais de 3 telas"	"ressaltou" ausente em <code>_ATtribution_re</code> → SpaCy etiquetava "Amanda" como ORG → <code>_is_org_context_ner</code> rejeitava incorretamente
25	Falso positivo	"o MVP contemplará apenas agendamento presencial"	"apenas" acionava <code>_BR_PrecHECK</code> ; decisão de escopo não filtrada

Soluções implementadas:

1. `_ATtribution_re` — adicionados verbos de atribuição ausentes: `ressaltou`, `acrescentou`, `ênfatizou`, `afirmou`, `reforçou`, `pontuou`. Com "ressaltou" reconhecido, `_extract_core_clause` remove "Amanda ressaltou que" antes do NER, eliminando o falso rejeite.
2. `_ORGANIZATIONAL_RE` — dois novos padrões:
 - `objetivo (principal|geral) + lançar/criar/implementar` — rejeita declarações de objetivo organizacional

- o `MVP + contempla/inclui/apenas/sem módulo` — rejeita decisões de escopo de MVP

3. `_NF_PROHIBITION` + `_NF_STRONG` — padrão `não pode ter mais de \d+` adicionado a ambos:

- o `_NF_PROHIBITION` impede que "não pode" acione retorno antecipado como BR
- o `_NF_STRONG` garante classificação como NF mesmo se o SVM retornar `uncertain`

Resultado: Clínica 22/27 → 25/27 (92.6%) · Agregado 56/61 (91.8%), F1=0.92

Próximos Passos — Longo Prazo (alto esforço, alto impacto)

7. XLM-RoBERTa multilíngue

Modelo fine-tuned em 100 idiomas incluindo PT e EN. Resolve definitivamente o problema multilíngue com um único modelo robusto. Accuracy esperada: >95% em ambos os idiomas.

8. Active Learning — o modelo aprende com o uso

Permitir que o BA marque classificações incorretas na UI. O sistema salva as correções e retreina periodicamente com os dados reais do cliente. O modelo melhora com o uso.

9. Score de Qualidade de Requisitos (IEEE 830)

Motivação

A classificação do tipo de um requisito (F/NF/BR) responde ao "o quê", mas não ao "quão bem escrito está". Um requisito classificado corretamente como Funcional pode ainda ser ambíguo, não mensurável ou composto — problemas que impactam diretamente a rastreabilidade e os testes. O IEEE 830 é o padrão mais reconhecido na literatura para avaliar a qualidade de especificações de requisitos.

Implementação

O score é calculado em `requirements_analyzer.py:score_requirement()` com 4 critérios binários (0 ou 1 ponto cada):

Critério	Heurística	Regex / lógica
Verbo de obrigação	Deve conter <code>deve</code> , <code>shall</code> , <code>must</code> , <code>will</code> , <code>should</code>	<code>\b(deve deverá shall must ...)\b</code>
Mensurabilidade	Deve conter número + unidade de medida	<code>\d+\s*(segundo ms % MB req ...)\b</code>
Ausência de ambiguidade	Não deve conter palavras vagas	Lista com 24 termos (rápido, adequado, sufficient, friendly...)
Requisito único	Não deve conter dois verbos de obrigação separados por "e/and"	<code>\b(deve shall)\b.{5,80}\b(e and)\b.{5,80}\b(deve shall)\b</code>

Score total: 0–4 pontos → rótulo e ícone:

Score	Rótulo	Ícone
4	Excelente	
3	Bom	
2	Regular	
1	Ruim	
0	Inválido	

Exemplos de Resultados

Requisito	Score	Label	Problemas
"O sistema deve processar 100 transações por segundo."	4/4	 Excelente	—
"O login deve ser seguro."	3/4	 Bom	Sem critério mensurável
"O sistema deve ser rápido e adequado para o usuário."	2/4	 Regular	Sem critério mensurável; palavras vagas: rápido, adequado
"Somente usuários cadastrados podem acessar o painel."	2/4	 Regular	Sem verbo de obrigação; sem critério mensurável
"O sistema deve autenticar e deve autorizar o usuário."	3/4	 Bom	Sem critério mensurável

Integração no Pipeline

O `score_requirement()` é chamado automaticamente dentro de `extract_requirements()` — nenhuma ação adicional do BA é necessária. O resultado é adicionado ao dict de cada requisito:

```
{
  'text': 'O sistema deve processar 100 transações por segundo.',
  'type': 'non_functional',
  'quality_score': 4,
  'quality_label': 'Excelente',
  'quality_icon': '',
  'quality_issues': [],
}
```

Na UI, o ícone e o label de qualidade aparecem abaixo de cada requisito na aba "Por Tipo". O resumo acima da lista exibe a distribuição de qualidade de todo o conjunto.

10. Agrupamento Temático e Detecção de Duplicatas

Motivação

Documentos de requisitos reais frequentemente contêm dezenas de itens sem organização temática explícita. Duas consequências práticas:

- O BA precisa reorganizar manualmente para identificar lacunas ou sobreposições
- Requisitos similares escritos de formas diferentes passam despercebidos, gerando implementação duplicada

10.1 Agrupamento Temático (K-means Semântico)

Implementação — requirements_analyzer.py:group_requirements()

```
Requisitos classificados
↓
Embedding via all-MiniLM-L6-v2
↓
KMeans (k automático = min(7, max(2, n//5)))
↓
Para cada cluster: extrair top-3 palavras por frequência
(stopwords PT+EN removidas)
↓
Rótulo do grupo = "Palavra1 / Palavra2 / Palavra3"
```

Determinação automática de k: `k = max(2, n // 5)` limitado a 7. Com 10 requisitos → 2 grupos; com 30 → 6 grupos; com 40+ → 7 grupos. A lógica evita grupos unitários e grupos muito heterogêneos.

Resultado em teste (6 requisitos):

Grupo	Rótulo gerado	Requisitos
0	Autenticar / Processar / Criptografar	Sistema deve autenticar; processar 100 req/s; criptografar dados; responder em 2s; recuperação de senha
1	Admin / Pode / Deletar	Somente admin pode deletar registros

O rótulo é gerado automaticamente e pode não ser semanticamente perfeito — serve como ponto de partida para organização, não como classificação definitiva.

Formato de saída: os campos `group_id` e `group_label` são adicionados ao dict de cada requisito. O agrupamento é visualizado na aba "Por Grupo" da UI, com expansores por cluster.

10.2 Detecção de Duplicatas

Implementação — requirements_analyzer.py:find_duplicates()

```
Todos os requisitos classificados
↓
Embedding via all-MiniLM-L6-v2
↓
Cosine similarity pairwise (matriz n×n)
↓
Filtrar pares com similarity ≥ 0.82
↓
Ordenar por similarity (maior primeiro)
↓
Exibir para revisão do BA
```

Threshold de 0.82: escolhido empiricamente para distinguir:

- Requisitos semanticamente idênticos escritos de forma diferente (>0.82)
- Requisitos do mesmo tema mas com conteúdo distinto (<0.82)

Exemplo de par detectado:

A	B	Similaridade
"O sistema deve autenticar o usuário com login e senha."	"O sistema deve fazer login do usuário usando credenciais."	0.83

Esses dois requisitos fariam da mesma funcionalidade e um deveria ser eliminado ou consolidado.

Resultado: a aba "Duplicatas" exibe cada par com barra de progresso de similaridade e os metadados de cada requisito (tipo + qualidade), permitindo ao BA decidir qual manter.

Decisão de design: análise pós-classificação separada

As três funções (`score_requirement` , `group_requirements` , `find_duplicates`) foram implementadas em `requirements_analyzer.py` separado do `requirements_extractor.py` . A razão é de responsabilidade: o extrator *classifica* cada sentença individualmente; o analisador *avalia o conjunto* completo. Manter a separação permite usar o analisador em outros contextos sem carregar o pipeline ML completo.

11. Avaliação Formal do Pipeline em Texto Corrido

Motivação

Os resultados apresentados na Seção 6 foram obtidos no conjunto de teste do dataset PROMISE NFR — sentenças já isoladas e pré-rotuladas. Em uso real, o pipeline recebe documentos completos com narrativa misturada a requisitos. Para medir o desempenho nesse cenário mais realista, foi conduzida uma avaliação formal com **ground truth anotado manualmente**.

Metodologia

Documento de teste: `documents/test_texto_corrido.txt` — ata de reunião fictícia do projeto "Portal do Cliente — Banco Nexus", com 34 sentenças após tokenização NLTK. O documento inclui narrativa, decisões de projeto, cabeçalhos, e requisitos embutidos em texto corrido (com e sem prefixos de atribuição como "Ana anotou que...", "Ricardo complementou dizendo que...").

Anotação: cada sentença recebeu manualmente um label de referência (*ground truth*):

- `irrelevant` — narrativa, contexto, cabeçalho, decisão de escopo (16 sentenças)
- `functional` — requisito funcional (4 sentenças)
- `non_functional` — requisito não-funcional (11 sentenças)
- `business_rule` — regra de negócio (3 sentenças)

Métricas calculadas em três etapas:

Etapa	O que mede
Etapa 1 — Filtro semântico	Precision/Recall/F1 para detectar se uma sentença é requisito (vs. irrelevante)
Etapa 2 — Classificação de tipo	Precision/Recall/F1 por classe (F, NF, BR), apenas nas sentenças que passaram o filtro
Etapa 3 — Pipeline completo	Precision/Recall/F1 considerando filtro correto e tipo correto simultaneamente

O script `src/evaluate_pipeline.py` automatiza a avaliação e exibe uma tabela linha a linha com `✓/✗` por sentença.

Resultado final (após todas as melhorias): 56/61 sentenças corretas — **91.8%, F1=0.92** em 2 domínios distintos. Detalhado na Seção 11.6. As seções 11.1 a 11.5 documentam o histórico de iterações que levou a esse resultado.

11.1 Resultados Iniciais — TF-IDF + SVM (sem desambiguação)

Etapa	Precision	Recall	F1
Filtro semântico	0.83	0.83	0.83
Classificação — Funcional	0.00	0.00	0.00
Classificação — Não-Funcional	0.50	0.64	0.56
Classificação — Regra de Negócio	0.50	0.33	0.40
Pipeline completo	0.73	0.44	0.55

Acurácia geral: 21/34 = **61.8%**

Diagnóstico dos erros identificados:

Categoria de erro	Qtd	Exemplos
Funcional classificado como NF	3	"portal deve enviar notificações push", "precisam conseguir fazer TED e PIX"
Falso positivo (irrelevante detectado)	3	Cabeçalho da ata, "banco precisa modernizar o canal digital"
Falso negativo (requisito não detectado)	3	"clientes querem poder configurar alertas", "mensagem de bloqueio seja clara"
NF classificado como BR	1	"não pode armazenar senhas em texto plano"
NF classificado como Incerto	2	"conformidade com a LGPD", "exportação deve ser registrada no log"

O problema estrutural mais relevante: o SVM **nunca prediz Funcional** neste documento (F1=0.00). A causa é o viés do PROMISE NFR — verbos de obrigação genéricos ("deve", "precisará") aparecem em ambas as classes com frequências similares, e o modelo não aprendeu a distinção semântica entre *executar uma funcionalidade* e *satisfazer um atributo de qualidade*.

11.2 Melhoria: Camada de Desambiguação por Sinais de Domínio

Para resolver o F1=0.00 de Funcional sem retreinar o modelo, foi implementada uma **camada de desambiguação pós-SVM** em `requirements_extractor.py:_disambiguate_fn_nf()`.

A abordagem usa feature engineering com padrões de domínio — técnica clássica de PLN que enriquece o sinal do modelo com conhecimento especializado:

Sinal	Padrão	Ação
Métrica quantitativa	<code>\d+ (segundos ms % tps MB...)</code>	Força NF
Conformidade/auditoria	LGPD, WCAG, ISO, PCI, criptograf, TLS, audit	Força NF
Proibição de qualidade	"não pode armazenar senha", "must not store password"	Força NF (era BR)
Verbo de ação transitivo	"deve mostrar/enviar/configurar/permitir/fazer..."	Força Funcional (se sem sinal NF)

Regras aplicadas em ordem de prioridade:

- 1. Proibição de qualidade → NF (corrige falsa BR)
- 2. Sinal forte de NF → NF (protege NFs corretas de virar Funcional)
- 3. Verbo de ação + sem sinal NF → Funcional (corrige o viés do SVM)
- 4. Sem sinal → mantém predição do SVM

11.3 Melhoria: Expansão de Regras de Negócio + Bypass do Filtro Semântico

Após a desambiguação, a análise de erros revelou dois problemas remanescentes nas Regras de Negócio:

- 1. **idx 11** — "limite padrão... R\$ 1.000 — acima disso o sistema deverá exigir confirmação por biometria" — falhava no **filtro semântico** (score abaixo de 0.35 mesmo com threshold adaptativo). O padrão condicional estava implícito no termo *"acima disso"*.

2. **idx 14** — "O desbloqueio só pode ser feito pelo atendimento humano" — falhava na **classificação** por ausência do padrão "só pode" no regex de BR.

Soluções implementadas:

Novos padrões adicionados ao regex de Regras de Negócio:

```
r'\bsó\s+pode[m]?\b' # restrição de permissão: "só pode ser feito por X"
r'\bacima\s+(disso|desse\s+valor)' # consequência de threshold: "acima disso o sistema deverá"
r'\b(valor|limite)\b.{3,60}\b(acima|ultrapassar|exceder)\b'
```

Bypass do filtro semântico para BR: sentenças com marcador BR explícito agora saltam o filtro semântico diretamente — se há um padrão condicional/restritivo claro, a sentença nunca é narrativa pura. Isso resolve casos onde a tradução automática da sentença produz um embedding distante dos âncoras PROMISE.

```
has_br_marker = bool(_BR_PRECHECK.search(s.lower()))
if not has_br_marker and not is_requirement_candidate(s):
    continue # só descarta se NÃO tem marcador BR E falha no filtro semântico
```

11.4 Evolução Completa — Documento Bancário (34 sentenças)

Etapas	Baseline SVM	+ Desambig.	+ BR expandida	+ Filtros contexto
Filtro semântico F1	0.83	0.83	0.86	0.91
Funcional F1	0.00	0.57	0.57	1.00
Não-Funcional F1	0.56	0.55	0.55	0.90
Regra de Negócio F1	0.40	0.40	1.00	0.80
Pipeline completo F1	0.55	0.60	0.69	0.91
Acurácia	61.8%	64.7%	70.6%	91.2%

11.5 Comparação Final: SVM vs. Zero-Shot

Para investigar se uma abordagem baseada em Transformers resolveria o problema de classificação de tipo, foi implementado um classificador **Zero-Shot** usando o modelo `cross-encoder/nli-deberta-v3-small` da Hugging Face — conteúdo diretamente coberto pela disciplina (aula 17/04: *Classificação de Textos: Naive Bayes baseline vs. Zero-Shot via Hugging Face*).

O Zero-Shot dispensa dados de treino supervisionado: classifica cada sentença por similaridade semântica com hipóteses textuais fornecidas pelo desenvolvedor. Foram testadas três configurações:

Hipóteses naïve (labels genéricos):


```
"functional software requirement"
"non-functional quality attribute"
"business rule or constraint"
```

Hipóteses refinadas (descritivas):

```
"the system must perform an action or provide a feature to the user"
"quality attribute such as performance, security, reliability, availability, compliance or accessibility"
"conditional business rule: if condition then obligation, or explicit prohibition or restriction"
```

Evolução Completa das Abordagens

Abordagem	Acurácia	F1 Filtro	F1 Funcional	F1 Completo
SVM — baseline	21/34 (61.8%)	0.83	0.00	0.55
Zero-Shot naïve	16/34 (47.1%)	0.83	—	0.25
Zero-Shot refinado	—	0.83	—	0.35
SVM + desambiguação	22/34 (64.7%)	0.83	0.57	0.60
Zero-Shot + desambiguação	23/34 (67.6%)	0.83	0.46	0.59
SVM + desambiguação + BR	24/34 (70.6%)	0.86	0.57	0.69
Zero-Shot + desambiguação + BR	25/34 (73.5%)	0.86	0.46	0.69
SVM + todos os filtros	31/34 (91.2%)	0.91	1.00	0.91
Zero-Shot + todos os filtros	27/34 (79.4%)	1.00	0.60	0.76

Análise

A terceira wave de melhorias (filtros de contexto + bypass de usabilidade) foi decisiva, eliminando todos os erros de falso positivo e corrigindo a classificação de tipo. O SVM atingiu **F1 Funcional = 1.00** e **precision = 1.00** — sem nenhum falso positivo.

Os 3 erros remanescentes (idx 8, 13, 33 — F1 completo 0.91) são falsos negativos em requisitos embutidos em discurso indireto com verbos de atribuição não cobertos (lembrar , levantar). Ver Seção 11.6 para a avaliação em dois domínios.

O Zero-Shot também melhorou (73.5% → **79.4%**), beneficiando-se dos filtros de contexto. Seus 7 erros remanescentes são todos de **classificação de tipo**: confunde performance/expiração com Funcional e auditoria com Regra de Negócio — falhas do raciocínio NLI genérico sem calibração de domínio.

Perfil das abordagens no documento bancário:

- **SVM**: P=1.00 / R=0.83 / F1=0.91 — zero falsos positivos; 3 FN em discurso indireto
- **Zero-Shot**: P=1.00 / R=0.61 / F1=0.76 — sem falsos positivos, mas 7 erros de tipo

O Zero-Shot corrigiu seus próprios erros estruturais com os filtros, mas não consegue resolver os erros de tipo sem retreinamento ou calibração adicional das hipóteses.

Por que o SVM supervisionado é o modelo principal?

O SVM foi treinado especificamente no PROMISE NFR, com exemplos reais de requisitos de software. Aprendeu padrões léxicos do domínio sem depender de internet para download de modelo (180MB). O Zero-Shot, mesmo com hipóteses bem elaboradas, aplica raciocínio NLI genérico sem calibração para terminologia de engenharia de requisitos. Os dois atingem F1 equivalente, mas o SVM tem menor custo de inferência e zero dependência de rede.

Este resultado confirma um princípio fundamental em PLN: **para domínios específicos com dados anotados disponíveis, modelos supervisionados com feature engineering de domínio tendem a ser equivalentes ou superiores a abordagens zero-shot com transformers modernos.**

Modelo ativo: TF-IDF + SVM + desambiguação + BR expandida + filtros de contexto (`USE_ZERO_SHOT = False`). O Zero-Shot permanece disponível em `requirements_extractor.py` para comparação.

Limitações identificadas na avaliação (documento bancário)

- 1. **Especificidade dos filtros:** `_DOCUMENT_HEADER_RE` e `_ORGANIZATIONAL_RE` dependem de padrões como "banco + modernizar" e "time de QA + acesso". Em documentos de outros domínios podem não cobrir todos os falsos positivos.
- 2. **Discurso indireto:** 3 requisitos (idx 8, 13, 33) embutidos em frases de discurso indireto (`"Ricardo lembrou que..."`, `"Ricardo levantou uma questão: se..."`) não são detectados — o pipeline exige que o verbo de obrigação esteja na cláusula principal.

11.6 Avaliação Multi-Domínio — Ground Truth Expandido

Metodologia

Para avaliar a generalização além do domínio bancário, o segundo documento de teste foi o `documents/test_ata_clinica.txt` — ata de levantamento de requisitos de uma plataforma de agendamento médico (Clínica MedVida). O documento tem **27 sentenças** anotadas manualmente, em domínio completamente diferente do bancário.

Documento	Domínio	Sentenças	Requisitos	Irrelevantes
<code>test_texto_corrido.txt</code>	Bancário (Portal do Cliente)	34	18	16
<code>test_ata_clinica.txt</code>	Clínica Médica (Agendamento)	27	16	11
Total		61	34	27

Resultados por Documento

Bancário (SVM + todos os filtros):

Etapa	P	R	F1
Filtro semântico	1.00	0.83	0.91
Funcional	1.00	1.00	1.00
Não-Funcional	1.00	0.82	0.90
Regra de Negócio	1.00	0.67	0.80
Pipeline completo	1.00	0.83	0.91

Acurácia: **31/34 (91.2%)** — os 3 erros remanescentes são falsos negativos em requisitos embutidos em discurso indireto (idx 8, 13, 33).

Clínica Médica (mesmo pipeline, domínio nunca visto):

Etapa	P	R	F1
Filtro semântico	0.94	1.00	0.97
Funcional	0.83	1.00	0.91
Não-Funcional	1.00	0.89	0.94
Regra de Negócio	0.67	1.00	0.80
Pipeline completo	0.94	0.94	0.94

Acurácia: **25/27 (92.6%)** — 2 erros remanescentes:

- idx 8 (tipo errado): "prontuário não pode ser acessado" → classificado como BR em vez de NF (caso de fronteira: restrição de acesso é ambígua entre as duas categorias)
- idx 22 (FP): "médicos precisam bloquear... hoje isso é feito manualmente" → classificado como Funcional (contexto de problema atual, não requisito do sistema)

Resultados Agregados (61 sentenças)

Abordagem	Acurácia	F1 Filtro	F1 Completo
SVM + todos os filtros	56/61 (91.8%)	0.94	0.92

Análise

O pipeline generalizou bem para um domínio completamente diferente sem nenhum ajuste: **92.6% de acurácia na clínica**, comparado a 91.2% no bancário (domínio treinado).

Os 2 erros remanescentes são casos de fronteira legítimos:

1. **Ambiguidade BR ↔ NF** (idx 8): "prontuário não pode ser acessado por funcionário que não seja o médico responsável" — restrição de acesso a dados de saúde. A literatura de ER classifica esse tipo de regra tanto como NF (segurança/acesso) quanto como BR (política de autorização), dependendo do contexto organizacional.

2. **Contexto de problema atual** (idx 22): "os médicos precisam bloquear períodos da agenda, e hoje isso é feito manualmente em planilha" — a segunda cláusula ("hoje... manualmente") caracteriza o problema atual, não um requisito. Difícil de detectar sem análise discursiva mais profunda.

O **F1 agregado de 0.92** confirma que o pipeline é robusto além do domínio de treinamento.

12. Resumo Executivo

Aspecto	Status
Upload PDF/TXT/DOCX	✔ Funcionando
Classificação F/NF — EN only	✔ NB: 86.8% / SVM bilingue: 87.8%
Deteção de Regras de Negócio	✔ Regex PT+EN
Suporte a Português (PT)	✔ Tradução PT→EN + dataset bilingue — avaliação formal: 91.8% em 2 domínios PT
Filtro semântico (texto corrido)	✔ Sentence Transformers + âncoras PROMISE, threshold 0.455 + adaptativo 0.35
Threshold de confiança	✔ 0.65 — itens incertos marcados  para revisão
Score de qualidade IEEE 830	✔ 4 critérios, score 0-4, label + issues por requisito
Agrupamento temático	✔ K-means semântico, k automático, rótulo por palavras-chave
Deteção de duplicatas	✔ Cosine similarity pairwise, threshold 0.82
Geração de User Stories	✔ Flan-T5 local
Export CSV/JSON	✔ Inclui tipo, qualidade, score, grupo, problemas
Desambiguação por sinais de domínio	✔ Feature engineering pós-SVM — F1 Funcional: 0.00 → 0.57
Expansão de BR + bypass filtro semântico	✔ "só pode", threshold implícito — F1 BR: 0.40 → 1.00
Filtros de contexto + bypass usabilidade	✔ Cabeçalhos, contexto org., linguagem de desejo — acurácia 70.6% → 91.2%
Avaliação formal com ground truth (bancário)	✔ 34 sentenças — SVM: F1=0.91; 3 FN em discurso indireto
Comparação SVM vs. Zero-Shot	✔ SVM: F1=0.91 / Zero-Shot: F1=0.76 (bancário); SVM preferido por precision e custo
Verbos reflexivos <code>_FUNCTIONAL_ACTION</code>	✔ "deve se integrar" →  Funcional (grupo <code>(?:se\s+)?</code> adicionado)
Regras de Negócio implícitas	✔ 7 novos padrões: "exige aprovação", "deve ser único", "acima de X%"
NER para contexto organizacional	✔ SpaCy <code>en_core_web_sm</code> detecta ORG/GPE como sujeito → rejeita
Avaliação multi-domínio (61 sentenças)	✔ Bancário 91.2% + Clínica 92.6% → agregado 91.8%, F1=0.92
Active Learning	✗ Não implementado
Fine-tuning com Transformers	✗ Não implementado